

Konstruowanie obiektów z użyciem Three.js

1 Wprowadzenie

Celem jest tworzenie obiektów za pomocą biblioteki Three.js.

Zobaczmy, jak utworzyć nowe geometrie siatki od podstaw. Przyjrzymy się także niektórym innym wsparciom, które three.js zapewnia do pracy z obiektami i materiałami.

Przykłady zakładają, że biblioteka jest załadowana jako moduł ES. W pliku HTML konfigurujemy `importmap`, a w kodzie używamy importów z `'three'` oraz `'three/addons/...'`:

```
<script type="importmap">
{
  "imports": {
    "three": "https://unpkg.com/three@0.184.0/build/three.module.js",
    "three/addons/": "https://unpkg.com/three@0.184.0/examples/jsm/"
  }
}
</script>
<script type="module">
  import * as THREE from 'three';
  // dalszy kod ...
</script>
```

2 Indeksowane zestawy ścian

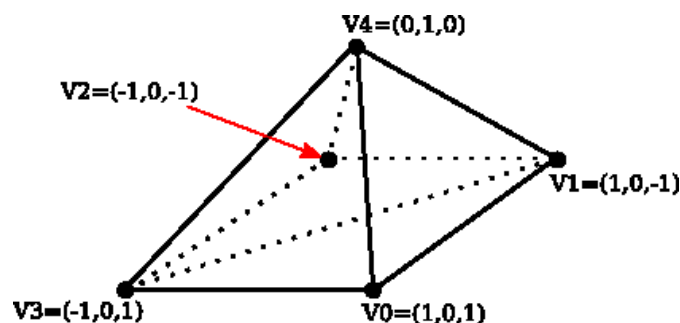
Siatka (*mesh*) w three.js jest indeksowanym zestawem ścian. W siatce three.js wszystkie wielokąty są trójkątami.

Geometria w three.js jest obiektem typu `THREE.BufferGeometry`. Dowolny obiekt geometrii przechowuje dane wierzchołków w postaci tzw. *atrybutów* — obiektów typu `THREE.BufferAttribute` opakujących tablice typowane (`Float32Array`, `Uint16Array` itd.). Najważniejszy atrybut nosi nazwę `position` i zawiera współrzędne wierzchołków: trzy kolejne liczby (x, y, z) opisują jeden wierzchołek.

Geometria może być *indeksowana* lub *nieindeksowana*. Geometria indeksowana dodatkowo posiada tablicę indeksów (ustawianą metodą `setIndex`), w której każda trójka liczb całkowitych wyznacza jedną ścianę trójkątną — trzy wierzchołki odczytywane są z tablicy `position` po wskazanych indeksach. Jako przykład zobaczmy, jak bezpośrednio utworzyć geometrię three.js dla tej piramidy:

Należy zauważyć, że dolna powierzchnia piramidy, która jest kwadratem, musi być podzielona na dwa trójkąty, aby piramida była reprezentowana jako geometria siatki. Mając to na uwadze, możemy zdefiniować:

```
const pyramidVertices = new Float32Array([
  // x, y, z      // numer wierzchołka
  1,  0,  1,      // 0
  1,  0, -1,      // 1
```



```

-1,  0, -1,    // 2
-1,  0,  1,    // 3
 0,  1,  0,    // 4
]);

const pyramidIndices = [
  3, 2, 1,    // pierwsza połowa dolnej ściany
  3, 1, 0,    // druga połowa dolnej ściany
  3, 0, 4,    // pozostałe ściany to cztery boki
  0, 1, 4,
  1, 2, 4,
  2, 3, 4
];

const pyramidGeom = new THREE.BufferGeometry();
pyramidGeom.setAttribute(
  'position',
  new THREE.BufferAttribute(pyramidVertices, 3)
);
pyramidGeom.setIndex(pyramidIndices);

```

Należy zauważyć, że **kolejność wierzchołków** w indeksach ściany nie jest całkowicie dowolna: powinny być wymienione w *kolejności przeciwnej do ruchu wskazówek zegara*, jak widać z przodu ściany, to znaczy patrząc na ścianę z zewnątrz piramidy.

Ta geometria piramidy, jak podano, będzie działać z `MeshBasicMaterial`, ale do pracy z podświetlonymi materiałami, takimi jak `MeshLambertMaterial`, `MeshPhongMaterial` czy `MeshStandardMaterial`, geometria wymaga atrybutu `normal` zawierającego wektory normalne. Jeśli geometria nie ma wektorów normalnych, materiały te pozostaną czarne. Możliwe jest ręczne zbudowanie atrybutu `normal`, ale możemy także obliczyć go automatycznie, wywołując metodę:

```
pyramidGeom.computeVertexNormals();
```

Dla geometrii indeksowanej metoda ta ustawia normalną każdego wierzchołka jako średnią normalnych tych ścian, które dzielą ten wierzchołek — otrzymujemy wtedy gładkie cieniowanie. Dla geometrii nieindeksowanej (gdy każdy wierzchołek pojawia się tylko w obrębie jednej ściany) każdej jego kopii przypisywana jest normalna tej właśnie ściany, co daje *płaskie cieniowanie*.

Aby uzyskać płaskie cieniowanie, mamy dwie równoważne drogi. Pierwsza to użyć materiału z włączoną opcją `flatShading`:

```

const material = new THREE.MeshStandardMaterial({
  color: 'gray',
  flatShading: true
});

```

Druga to zbudować geometrię *nieindeksowaną* — każdy wierzchołek jest wtedy zduplikowany tyle razy, ile ścian go współdzieli. Do konwersji służy metoda `toNonIndexed()`:

```
const flatGeom = pyramidGeom.toNonIndexed();
flatGeom.computeVertexNormals(); // normalne ścian w atrybucie wierzchołków
```

Standardowe geometrie three.js, takie jak `BoxGeometry`, mają prawidłowe atrybuty `position`, `normal` oraz `uv` ustawione automatycznie.

Dzięki pełnemu zestawowi normalnych piramida jest gotowa do użycia z dowolnymi materiałami siatkowymi, ale wygląda trochę nudno z jednym kolorem. Możliwe jest użycie kilku kolorów na jednej siatce. Najprościej zrobić to, dostarczając tablicę materiałów do konstruktora obiektu siatki zamiast pojedynczego materiału. Każda ściana geometrii musi wówczas mieć przypisany indeks materiału (*material group*). Standardowe geometrie tworzą odpowiednie grupy same, np. aby zrobić sześcián z różnymi materiałami na sześciu ścianach:

```
const cubeGeom = new THREE.BoxGeometry(10, 10, 10);
const cubeMaterials = [
  new THREE.MeshPhongMaterial({ color: 'red'    }), // ściana +x
  new THREE.MeshPhongMaterial({ color: 'cyan'   }), // ściana -x
  new THREE.MeshPhongMaterial({ color: 'green'  }), // ściana +y
  new THREE.MeshPhongMaterial({ color: 'magenta' }), // ściana -y
  new THREE.MeshPhongMaterial({ color: 'blue'   }), // ściana +z
  new THREE.MeshPhongMaterial({ color: 'yellow'  }) // ściana -z
];
const cube = new THREE.Mesh(cubeGeom, cubeMaterials);
```

Dla geometrii tworzonej ręcznie odpowiednie grupy możemy zdefiniować metodą `addGroup(start, count, materialIndex)`, gdzie `start` to indeks pierwszego wykorzystanego wpisu w tablicy indeksów (lub w atrybucie `position` dla geometrii nieindeksowanej), a `count` to liczba zużytych wpisów (3 na trójkąt).

Drugim sposobem zastosowania kolorów jest przypisanie koloru każdemu wierzchołkowi osobno. WebGL interpoluje wtedy kolory wierzchołków, aby obliczyć kolory pikseli wewnątrz ścian. Dane te przekazujemy przez atrybut `color` geometrii, a na materiale włączamy flagę `vertexColors`:

```
const colors = new Float32Array([
  1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, // cztery wierzchołki podstawy
  1, 0, 0                                // wierzchołek szczytowy
]);
pyramidGeom.setAttribute('color', new THREE.BufferAttribute(colors, 3));

const material = new THREE.MeshLambertMaterial({
  color: 'white',
  vertexColors: true
});
```

Składowe kolorów wierzchołków mnożone są składowo przez kolor materiału — dlatego ustawiamy go na biały, aby kolory geometrii nie zostały przyciemnione. Aby uzyskać efekt „różnych kolorów na różnych ścianach” przy geometrii nieindeksowanej, wystarczy w trzech wierzchołkach tworzących daną ścianę wpisać ten sam kolor.

3 Krzywe i powierzchnie

Oprócz umożliwienia tworzenia indeksowanych zestawów ścian, three.js obsługuje także krzywe i powierzchnie zdefiniowane matematycznie.

Powierzchnie parametryczne są najłatwiejsze do pracy. Powierzchnia parametryczna jest zdefiniowana przez funkcję matematyczną $f(u, v)$, gdzie u i v są liczbami, a każda wartość funkcji jest punktem w przestrzeni. Powierzchnia składa się ze wszystkich punktów, które są wartościami funkcji dla u i v w niektórych określonych zakresach. W three.js funkcja jest zwykłą funkcją JavaScript, która *zapisuje* obliczony wynik do przekazanego jej obiektu `THREE.Vector3` — jest to zmiana względem wcześniejszego API (nie `return`, tylko parametr wyjściowy `target`). Wartości u i v są zawsze w zakresie od 0,0 do 1,0.

Klasa `ParametricGeometry` jest obecnie dodatkiem (*addon*), więc importujemy ją osobno:

```
import { ParametricGeometry }
  from 'three/addons/geometries/ParametricGeometry.js';
```

Geometria jest tworzona przez konstruktor

```
new ParametricGeometry( func, slices, stacks )
```

gdzie `func` jest funkcją JavaScript, a `slices` i `stacks` określają liczbę podpodziałów przedziału od 0 do 1 odpowiednio w kierunku u i v . Po uzyskaniu geometrii możesz użyć jej do utworzenia siatki w zwykły sposób. Oto przykład. Ta powierzchnia jest zdefiniowana przez funkcję

```
function surfaceFunction(u, v, target) {
  // u i v są w zakresie od 0 do 1.
  const x = 20 * (u - 0.5);           // x i z w zakresie od -10 do 10
  const z = 20 * (v - 0.5);
  const y = 2 * (Math.sin(x / 2) * Math.cos(z));
  target.set(x, y, z);
}
```

a siatka three.js reprezentująca powierzchnię jest tworzona za pomocą

```
const surfaceGeometry = new ParametricGeometry(surfaceFunction, 64, 64);
const surface = new THREE.Mesh(surfaceGeometry, material);
```

Krzywe są bardziej skomplikowane w three.js. Klasa `THREE.Curve` reprezentuje abstrakcyjną koncepcję krzywej parametrycznej w dwóch lub trzech wymiarach. (Nie reprezentuje geometrii three.js.) Krzywa parametryczna jest zdefiniowana przez funkcję jednej zmiennej numerycznej t . Metoda `curve.getPoint(t, optionalTarget)` powinna zwrócić punkt na krzywej odpowiadający wartości parametru t — typu `THREE.Vector2` dla krzywej 2D lub `THREE.Vector3` dla krzywej 3D. Aby uzyskać rzeczywistą krzywą, musimy zdefiniować własną podklasę `Curve`. Na przykład:

```
class Helix extends THREE.Curve {
  getPoint(t, optionalTarget = new THREE.Vector3()) {
    const s = (t - 0.5) * 12 * Math.PI;
    // gdy t zmienia się od 0 do 1, s zmienia się od -6 do 6
    return optionalTarget.set(
      5 * Math.cos(s),
      s,

```

```

        5 * Math.sin(s)
    );
}
}
const helix = new Helix();

```

Po zdefiniowaniu `getPoint` mamy użyteczną krzywą. Jedną z rzeczy, które można z tym zrobić, jest utworzenie geometrii rury, która definiuje powierzchnię będącą rurą o okrągłym przekroju poprzecznym, a krzywa biegnie wzdłuż środka rury. Przykładowy program używa zdefiniowanej powyżej krzywej `helix` do utworzenia dwóch rur (rys. 1). Geometria szerszej rury jest tworzona za pomocą

```
const tubeGeometry1 = new THREE.TubeGeometry(helix, 128, 2.5, 32);
```

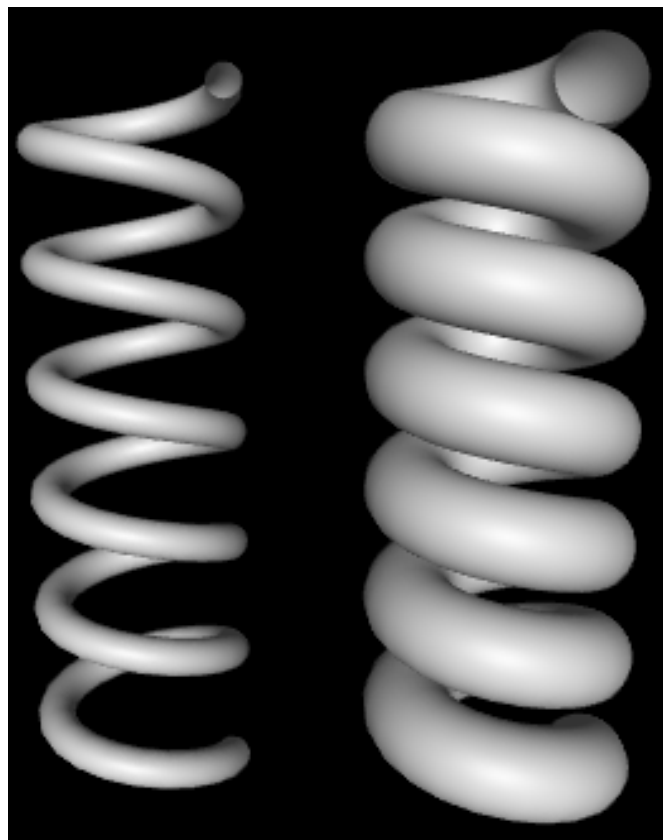


Figure 1: `THREE.TubeGeometry`

Drugim parametrem konstruktora jest liczba podziałów powierzchni wzdłuż długości krzywej. Trzeci jest promieniem kołowego przekroju poprzecznego rury, a czwarty jest liczbą podziałów na obwodzie przekroju poprzecznego.

Istnieje również kilka sposobów tworzenia powierzchni z krzywej 2D. Jednym ze sposobów jest obrócenie krzywej wokół linii, generowanie powierzchni obrotu. Powierzchnia składa się ze wszystkich punktów, przez które przechodzi krzywa, gdy się obraca. Jest to tak zwane toczenie (**lathing**). Rysunek 2 pokazuje powierzchnię wygenerowaną przez toczenie krzywej kosinusoidalnej. (Obraz jest obrócony o 90 stopni, tak że oś Y jest pozioma.) Krzywa jest pokazana powyżej powierzchni.

Powierzchnia jest tworzona w `three.js` za pomocą obiektu `THREE.LatheGeometry`. Geometria `LatheGeometry` jest zbudowana nie z krzywej, ale z tablicy punktów leżących na krzywej. Punkty

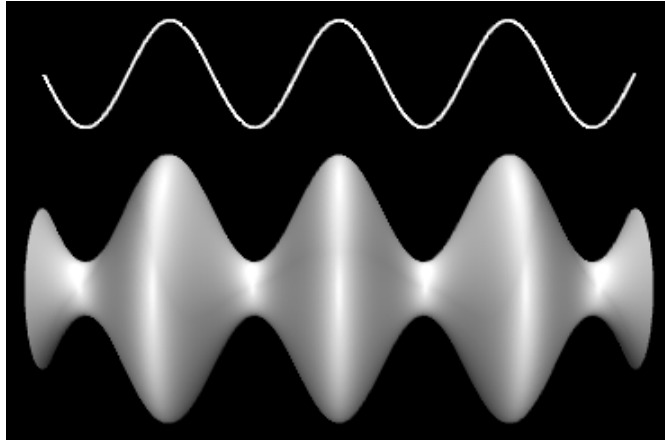


Figure 2: THREE.LatheGeometry

są obiektami typu `Vector2`, a krzywa leży w płaszczyźnie xy . Powierzchnia jest generowana przez obrót krzywej wokół osi y . Konstruktor `LatheGeometry` przyjmuje postać

```
new THREE.LatheGeometry( points, segments )
```

Pierwszym parametrem jest tablica `Vector2`. Druga to liczba podziałów powierzchni wzdłuż okręgu generowanego, gdy punkt jest obracany wokół osi. (Liczba „stosów” powierzchni jest określona przez długość tablicy punktów.)

Kolejną rzeczą, którą możesz zrobić za pomocą krzywej 2D, jest wypełnienie wnętrza krzywej, dając kształt wypełniony 2D. Aby to zrobić w `three.js`, można użyć obiektu typu `THREE.Shape`, który jest podklasą `THREE.Path` (a ta z kolei dziedziczy po `THREE.Curve`). Kształt można zdefiniować w taki sam sposób, jak ścieżkę w interfejsie API 2D Canvas. Oznacza to, że obiekt typu `THREE.Shape` ma metody `shape.moveTo`, `shape.lineTo`, `shape.quadraticCurveTo` i `shape.bezierCurveTo`, których można użyć do zdefiniowania ścieżki. Jako przykład możemy utworzyć kształt łyży:

```
const path = new THREE.Shape();
path.moveTo(0, 10);
path.bezierCurveTo( 0, 5, 20, -10, 0, -10);
path.bezierCurveTo(-20, -10, 0, 5, 0, 10);
```

Aby użyć ścieżki do utworzenia wypełnionego kształtu w `three.js`, potrzebujemy obiektu `ShapeGeometry`:

```
const shapeGeom = new THREE.ShapeGeometry(path);
```

Jeśli zamiast płaskiego kształtu chcemy uzyskać bryłę, tę samą ścieżkę można przekazać do `THREE.ExtrudeGeometry`, która wyciąga profil wzdłuż zadanej głębokości i opcjonalnie dodaje fazowanie krawędzi.

4 Ładowanie modeli

`Three.js` nie używa już własnego formatu modeli opartego na JSON. Standardem wymiany zasobów 3D na stronach internetowych jest dziś **glTF** (rozszerzenia `.gltf` i `.glb`). Format ten jest binarnie zwarty, opisuje kompletną scenę (geometrie, materiały, tekstury, animacje, szkielety, kamery i światła) i jest eksportowany bezpośrednio z Blendera, 3ds Max i innych narzędzi.

Do jego ładowania służy klasa `GLTFLoader`, dostępna jako dodatek:

```
import { GLTFLoader } from 'three/addons/loaders/GLTFLoader.js';
```

Jeśli `loader` jest obiektem typu `GLTFLoader`, możesz użyć jego metody `load()`, aby rozpocząć proces ładowania modelu:

```
loader.load(url, onLoad, onProgress, onError);
```

Pierwszy parametr to adres URL pliku zawierającego model. Drugi parametr, `onLoad`, to funkcja, która zostanie wywołana po zakończeniu ładowania. Ładowanie jest asynchroniczne; `loader.load()` uruchamia proces i natychmiast powraca. Obowiązkiem funkcji zwrotnej jest wykorzystanie danych z pliku do dodania obiektu do sceny. Funkcja zwrotna otrzymuje jeden parametr — obiekt `gltf`, którego właściwość `gltf.scene` jest gotowym `THREE.Group` zawierającym kompletny model wraz z materiałami i teksturami. Dodatkowo `gltf.animations` zawiera tablicę klipów animacyjnych.

Tutaj jest para funkcji, które mogą być użyte do załadowania modelu glTF z określonego adresu URL i dodania go do sceny:

```
function loadModel(url) {          // wywołaj tę funkcję, aby załadować model
    const loader = new GLTFLoader();
    loader.load(url, modelLoaded);
}

function modelLoaded(glTF) {      // callback ładowania
    const object = glTF.scene;
    scene.add(object);
    render(); // potrzebne tylko wtedy, gdy nie działa pętla animacji
}
```

Alternatywnie można skorzystać z wersji promise'owej `loadAsync`, co pozwala użyć `await` i lepiej obsłużyć błędy:

```
async function loadModel(url) {
    const loader = new GLTFLoader();
    const glTF = await loader.loadAsync(url);
    scene.add(glTF.scene);
    render();
}
```

Istnieją także inne ładowarki o zbliżonym interfejsie (np. `OBJLoader`, `FBXLoader`, `ColladaLoader`), ale w ramach tego laboratorium przyjmujemy glTF jako format domyślny.

Literatura

Uwaga!

Tworzenie aplikacji z wykorzystaniem `three.js`
<https://threejs.org/manual/#pl/fundamentals>
W języku angielskim

- oficjalny podręcznik: *Three.js Manual*
<https://threejs.org/manual/>
- dokumentacja API: <https://threejs.org/docs/>
- książka interaktywna: *Building Objects*
<http://math.hws.edu/graphicsbook/c5/s2.html> (rozdziały 5.2–5.3)

5 Zadanie

Celem jest konstruowanie modelu figury szachowej zgodnie z wariantem zadania (patrz rys. 3) używając three.js w oparciu o omówione na zajęciach metody konstruowania obiektów.



Figure 3: Warianty zadania